



UNIVERSIDAD TECNOLÓGICA DE PUEBLA INGENIERÍA EN DESARROLLO DE SOFTWARE MULTIPLATAFORMA

APLICACIÓN MULTIPLATAFORMA

REPORTE — SUBPRODUCTO NO. 5

**ASIGNATURA: APLICACIONES WEB
PROGRESIVAS**

**DOCENTE: MATHER XÓCHITL MENDOZA
PÍSCIL**

**ALUMNO: PÉREZ ALCÁNTARA ALEJANDRO
10° “D”**

FECHA: 28 DE SEPTIEMBRE DE 2026

ÍNDICE

1. Introducción.....	3
2. ¿Qué es una Aplicación Multiplataforma?.....	3
2.1 Tipos de Enfoques Multiplataforma.....	3
3. Ilustración: ¿Cómo funciona una Aplicación Multiplataforma?	4
4. Características Principales	5
5. Ventajas y Desventajas	5
6. Comparativa: Multiplataforma vs. Móvil vs. Web vs. Escritorio	6
7. Tecnología Utilizada	7
8. Ejemplo de Funcionamiento: ExpenseTrack Lite	8
9. Tabla Comparativa: Requisitos de una Aplicación Multiplataforma vs. ExpenseTrack Lite.....	17
10. Lista de Cotejo.....	18
11. Conclusión	19
12. URL de la Aplicación	19
Referencias.....	20

1. Introducción

A lo largo de esta materia he documentado los distintos enfoques de desarrollo de software: en el Subproducto No. 1 documenté el desarrollo nativo, usando como ejemplo propio mi proyecto de reloj inteligente en Wear OS; en el Subproducto No. 2 documenté el desarrollo web, usando como ejemplo propio MediCore Professional System; en el Subproducto No. 3 documenté el desarrollo de escritorio, usando como ejemplo propio TaskDesk; y en el Subproducto No. 4 documenté el desarrollo móvil, usando como ejemplo propio PediatriTrack Global. En este quinto reporte cierro el panorama con el enfoque de aplicaciones multiplataforma: programas que, a partir de un solo código fuente, se empaquetan y distribuyen para correr en distintos sistemas operativos de escritorio (Windows, macOS y Linux) sin tener que reescribir la lógica ni la interfaz para cada uno.

A lo largo del documento defino el concepto de aplicación multiplataforma, ilustro su funcionamiento mediante un diagrama, listo sus características principales, sus ventajas y desventajas, la comparo frente a los otros enfoques que ya documenté en esta materia (móvil, web, escritorio), documento la tecnología utilizada y, como evidencia propia, presento ExpenseTrack Lite: una aplicación de control de gastos personales que construí con Electron, un framework que empaqueta una interfaz web (HTML, CSS y JavaScript) junto con Node.js y Chromium para que corra como app de escritorio nativa en Windows, macOS y Linux desde el mismo código. Cierro el documento con una tabla comparativa entre lo que exige la definición de una aplicación multiplataforma y lo que realmente implementé, una lista de cotejo de autoevaluación y la URL del repositorio donde está el código.

2. ¿Qué es una Aplicación Multiplataforma?

Una aplicación multiplataforma es un programa de software que se desarrolla una sola vez, a partir de una única base de código, y se compila o empaqueta para poder ejecutarse en más de un sistema operativo o tipo de dispositivo, conservando la misma interfaz y lógica de negocio en cada uno. A diferencia de una aplicación nativa (que exige un código independiente por sistema operativo, como documenté en el Subproducto No. 1), la aplicación multiplataforma prioriza compartir el máximo código posible entre plataformas, a cambio de depender de una capa intermedia —un motor o runtime— que traduce ese código único a algo que cada sistema operativo puede ejecutar.

Esto conecta con lo que ya documenté en los subproductos anteriores: en el Subproducto No. 4 identifiqué que React Native (el framework de PediatriTrack Global) es, en realidad, un caso de desarrollo multiplataforma aplicado específicamente a dispositivos móviles. En este reporte extendiendo esa misma idea al escritorio, con un enfoque distinto: Electron, que en vez de compilar a componentes nativos como React Native, empaqueta un motor de navegador (Chromium) y un entorno de Node.js junto con la aplicación, de modo que el mismo código HTML/CSS/JS corre igual en Windows, macOS y Linux.

2.1 Tipos de Enfoques Multiplataforma

Identifiqué tres formas principales de lograr que una misma aplicación corra en varios sistemas operativos:

- **Motor de navegador embebido (Electron, Tauri):** empaquetan una interfaz web (HTML/CSS/JS) junto con un runtime (Chromium + Node.js en el caso de Electron) dentro de un ejecutable de escritorio; es el enfoque que uso en ExpenseTrack Lite.
- **Compilación a UI nativa desde un solo código (Flutter, React Native, .NET MAUI):** el mismo código fuente se traduce en tiempo de compilación a componentes de interfaz nativos de cada plataforma, sin depender de un navegador embebido.

- **Máquina virtual o bytecode compartido (Java/JVM, .NET tradicional):** el código se compila a un formato intermedio que corre sobre una máquina virtual instalada en cada sistema operativo, en vez de compilarse directamente a código nativo de cada uno.

3. Ilustración: ¿Cómo funciona una Aplicación Multiplataforma?

El siguiente diagrama ilustra el flujo general de una aplicación multiplataforma construida con un motor tipo Electron: desde que existe un único código fuente, hasta que ese mismo código se empaqueta en builds independientes para cada sistema operativo, y cada build corre como una app de escritorio nativa en la máquina del usuario final.

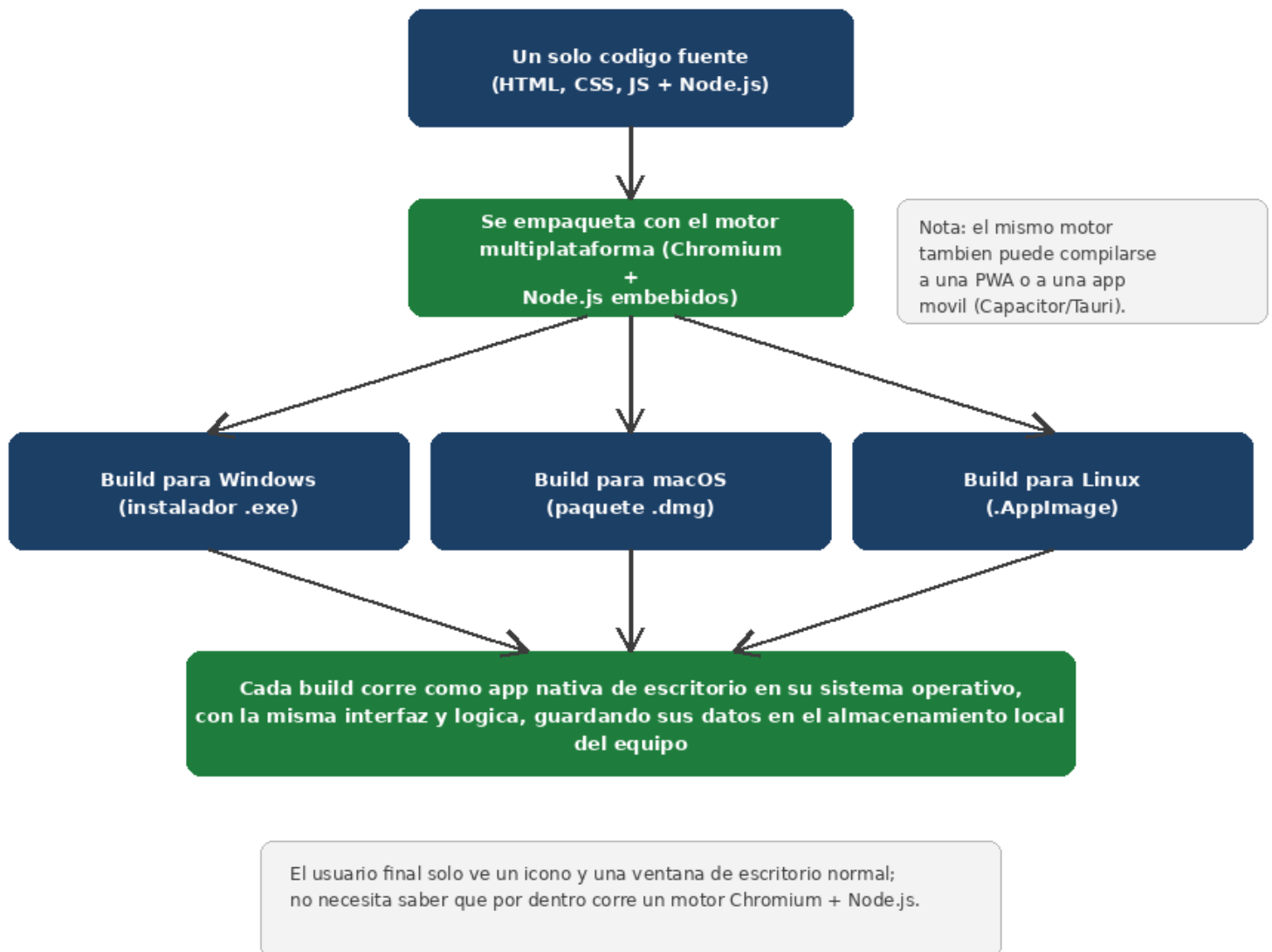


Figura 1. Flujo de funcionamiento de una aplicación multiplataforma: de un solo código fuente a builds independientes para Windows, macOS y Linux, cada uno corriendo como app nativa de escritorio.

4. Características Principales

- **Un solo código fuente compartido:** la interfaz y la lógica de negocio se escriben una sola vez y se reutilizan para todas las plataformas de destino.
- **Depende de un motor o runtime intermedio:** un componente (Chromium+Node.js, la máquina virtual de Flutter, el runtime de .NET, etc.) traduce ese código único a algo ejecutable en cada sistema operativo.
- **Se distribuye en builds independientes por sistema operativo:** un instalador .exe para Windows, un paquete .dmg para macOS y un .ApplImage o .deb para Linux, generados a partir del mismo proyecto.
- **Acceso al sistema de archivos y hardware del equipo:** a diferencia de una aplicación web pura, puede leer y escribir archivos locales, acceder a notificaciones del sistema y correr sin conexión a internet.
- **Consistencia visual entre plataformas:** la misma interfaz se ve prácticamente igual en Windows, macOS y Linux, lo que reduce el trabajo de diseño y pruebas frente a mantener interfaces nativas separadas.
- **Costo en tamaño y consumo de recursos:** al empaquetar su propio runtime (por ejemplo, Chromium completo en Electron), el instalador y el consumo de memoria suelen ser mayores que los de una app nativa equivalente.

5. Ventajas y Desventajas

A partir de las características anteriores, resumo las principales ventajas y desventajas del desarrollo multiplataforma, tal como hice en los subproductos anteriores para los otros enfoques:

VENTAJAS	DESVENTAJAS
Un solo código base reduce tiempo y costo de desarrollo frente a mantener una app nativa por sistema operativo.	El instalador y el consumo de memoria/CPU suelen ser mayores, al empaquetar su propio runtime.
Los cambios y correcciones se hacen una sola vez y se reflejan en todas las plataformas al reconstruir.	El acceso a funciones muy específicas del sistema operativo puede ser más limitado o requerir módulos adicionales.
El equipo de desarrollo puede ser más pequeño, ya que no necesita especialistas por cada plataforma.	El arranque de la aplicación suele ser un poco más lento que el de una app nativa equivalente.
Funciona sin conexión a internet, con acceso al sistema de archivos y almacenamiento local del equipo.	Depende de que el framework (Electron, Flutter, etc.) siga manteniéndose y actualizándose a futuro.

6. Comparativa: Multiplataforma vs. Móvil vs. Web vs. Escritorio

Para cerrar el panorama de los enfoques que documenté a lo largo de esta materia, comparo aquí el desarrollo multiplataforma frente a los otros tres que ya presenté: móvil (Subproducto No. 4, PediatriTrack Global), web (Subproducto No. 2, MediCore) y escritorio nativo (Subproducto No. 3, TaskDesk):

CRITERIO	MULTIPLATAFORMA	MÓVIL	WEB	ESCRITORIO NATIVO
Lenguaje	HTML, CSS, JS + Node.js (Electron)	JavaScript/TypeScript (React Native), Kotlin/Swift	HTML, CSS, JS (+ backend)	Python, C#, C++, Java
Rendimiento	Medio (mayor consumo por el runtime embebido)	Alto	Medio-bajo	Alto
Acceso al hardware/SO	Amplio (sistema de archivos, notificaciones)	Amplio (cámara, GPS, sensores)	Limitado (APIs del navegador)	Amplio (archivos, periféricos)
Instalación	Sí, un instalador por sistema operativo	Sí, desde tienda de apps	No requiere, se accede por URL	Sí (o requiere runtime)
Funciona sin internet	Sí	Sí, parcialmente	Muy limitado	Sí
Costo de desarrollo	Bajo-medio (1 código, 3 sistemas operativos)	Medio (React Native: 1 código, 2 plataformas)	Bajo (1 solo código)	Medio-alto (1 código por SO)
Ejemplos	ExpenseTrack Lite, VS Code, Discord	PediatriTrack Global, WhatsApp	MediCore, Google Docs, PWA	TaskDesk, mi app Wear OS

El desarrollo multiplataforma comparte con el móvil la idea central de un solo código para varios sistemas, pero aplicada al escritorio en vez de a Android/iOS; y comparte con el escritorio nativo (TaskDesk, hecho con Tkinter) el tipo de interfaz —una ventana de escritorio con controles, formularios y listas—, aunque a costa de un mayor consumo de recursos por depender de un runtime embebido. Elegí Electron para ExpenseTrack Lite precisamente por ese equilibrio: reutilizar mis habilidades de desarrollo web (HTML, CSS, JavaScript) para producir una app de escritorio real, instalable en Windows, macOS y Linux desde el mismo proyecto.

7. Tecnología Utilizada

Para ExpenseTrack Lite utilicé la siguiente tecnología:

COMPONENTE	TECNOLOGÍA	FUNCIÓN
Framework multiplataforma	Electron 31	Empaqueta la interfaz web junto con Chromium y Node.js como app de escritorio para Windows, macOS y Linux
Proceso principal (main)	Node.js (main.js)	Crea la ventana de la app, gestiona el ciclo de vida y expone los canales de comunicación (IPC)
Interfaz (renderer)	HTML, CSS y JavaScript	Formulario para registrar gastos, lista de gastos y cálculo del total, igual en las tres plataformas
Comunicación segura	contextBridge + ipcRenderer/ipcMain (preload.js)	Conecta la interfaz con el proceso principal sin exponer Node.js completo al renderer (contextIsolation); expone listar, agregar, editar, eliminar y exportarCSV
Almacenamiento local	Sistema de archivos (JSON en la carpeta de datos del usuario)	Guarda, edita y elimina los gastos localmente en el equipo, sin necesidad de servidor ni conexión a internet
Exportación de datos	Módulo fs de Node.js (app.getPath('documents'))	Genera un archivo .csv con todos los gastos en la carpeta de Documentos del sistema operativo del usuario
Empaquetado / distribución	electron-builder	Genera el instalador .exe (Windows), el paquete .dmg (macOS) y el .ApplImage (Linux) desde el mismo código
Control de versiones	Git y GitHub	Repositorio del proyecto con el historial de cambios del código

Elegí Electron porque me permite construir la interfaz con las mismas herramientas que ya usé en el Subproducto No. 2 (HTML, CSS y JavaScript), pero obteniendo una aplicación de escritorio real: con su propio icono, su propia ventana nativa, acceso al sistema de archivos del equipo y capacidad de funcionar sin conexión a internet, algo que una aplicación web pura no puede ofrecer. El proceso principal (main.js) y el proceso de la interfaz (renderer) se comunican mediante IPC a través de un script preload con contextIsolation activado, que es la forma recomendada y más segura de exponer funciones de Node.js a la interfaz sin arriesgar la seguridad de la aplicación. La función de exportar a CSV, en particular, aprovecha app.getPath('documents') de Electron, que devuelve automáticamente la ruta correcta de la carpeta de Documentos sin importar si la app corre en Windows, macOS o Linux — un detalle pequeño pero representativo de lo que significa programar pensando en varias plataformas a la vez.

8. Ejemplo de Funcionamiento: ExpenseTrack Lite

Como ejemplo propio de aplicación multiplataforma, presento ExpenseTrack Lite: una aplicación de escritorio para el control de gastos personales, que permite registrar un gasto (concepto, categoría y monto), editarlo, eliminarlo, filtrar la lista por categoría, y exportar todos los gastos a un archivo CSV. Toda la información se guarda localmente en el equipo, en un archivo JSON dentro de la carpeta de datos del usuario, sin depender de internet ni de un servidor externo.

La construí con Electron siguiendo la arquitectura recomendada: un proceso principal (main.js) que crea la ventana y gestiona los datos, un proceso de interfaz (index.html + renderer.js + style.css) y un script preload.js que conecta ambos de forma segura mediante contextBridge. A continuación documento con capturas reales el funcionamiento de la aplicación, corriendo aquí mismo como una ventana de escritorio nativa, incluyendo sus tres funciones más recientes: edición de gastos, filtro por categoría y exportación a CSV.

ExpenseTrack Lite
Control de gastos personales (100% local, sin internet)

Nuevo gasto

Concepto (ej. Comida, transporte...)

Comida ▼

Monto (MXN)

Agregar gasto

Mis gastos Total: \$0.00

Plataforma: linux — Electron 31.7.7 — Chromium 126.0.6478.234

Figura 2. Ventana inicial de ExpenseTrack Lite recién abierta: formulario para registrar un nuevo gasto y lista de gastos vacía, con el total en \$0.00.

ExpenseTrack Lite

Control de gastos personales (100% local, sin internet)

Nuevo gasto

Tacos con amigos

Comida

180

Agregar gasto

Mis gastos

Total: \$0.00

Plataforma: linux — Electron 31.7.7 — Chromium 126.0.6478.234

Figura 3. Formulario con un gasto capturado ("Tacos con amigos", categoría Comida, monto \$180) antes de presionar el botón "Agregar gasto".

ExpenseTrack Lite

Control de gastos personales (100% local, sin internet)

Nuevo gasto

Concepto (ej. Comida, transporte...)

Comida

▼

Monto (MXN)

Agregar gasto

Mis gastos

Total: \$180.00

Tacos con amigos

Comida • 2026-09-22

\$180.00

Eliminar

Plataforma: linux — Electron 31.7.7 — Chromium 126.0.6478.234

Figura 4. Primer gasto ya agregado a la lista, con el total actualizado automáticamente a \$180.00 y el pie de la ventana mostrando la plataforma y versión de Electron/Chromium en la que corre.

Página 10 de 20

ExpenseTrack Lite

Control de gastos personales (100% local, sin internet)

Nuevo gasto

Concepto (ej. Comida, transporte...)

Escuela

▼

Monto (MXN)

Agregar gasto

Mis gastos

Total: \$285.50

Tacos con amigos	\$180.00	Eliminar
Comida • 2026-09-22		
Camion a la universidad	\$45.50	Eliminar
Transporte • 2026-09-22		
Fotocopias e impresiones	\$60.00	Eliminar
Escuela • 2026-09-22		

Plataforma: linux — Electron 31.7.7 — Chromium 126.0.6478.234

Figura 5. Lista con tres gastos registrados de distintas categorías (Comida, Transporte, Escuela) y el total acumulado recalculado a \$285.50.

Página 11 de 20

ExpenseTrack Lite

Control de gastos personales (100% local, sin internet)

Nuevo gasto

Concepto (ej. Comida, transporte...)

Escuela

▼

Monto (MXN)

Agregar gasto

Mis gastos

Total: \$105.50

Camion a la universidad

Transporte • 2026-09-22

\$45.50

Eliminar

Fotocopias e impresiones

Escuela • 2026-09-22

\$60.00

Eliminar

Plataforma: linux — Electron 31.7.7 — Chromium 126.0.6478.234

Figura 6. Resultado tras eliminar el primer gasto de la lista ("Tacos con amigos"): quedan dos gastos y el total se recalcula a \$105.50.

Página 12 de 20

ExpenseTrack Lite

Control de gastos personales (100% local, sin internet)

Editar gasto

Tacos con amigos

Comida

▼

180

Guardar cambios

Cancelar edición

Mis gastos

Total: \$323.50

Filtrar por categoría

Todas

▼

Tacos con amigos

Comida • 2026-09-22

\$180.00

Editar

Eliminar

Camion a la universidad

Transporte • 2026-09-22

\$45.50

Editar

Eliminar

Fotocopias e impresiones

Escuela • 2026-09-22

\$60.00

Editar

Eliminar

Cafe con leche

Comida • 2026-09-22

\$38.00

Editar

Eliminar

Exportar a CSV

Figura 7. Función de edición: al presionar "Editar" sobre un gasto, el formulario cambia a modo "Editar gasto", se precarga con sus datos y el botón pasa a decir "Guardar cambios".

ExpenseTrack Lite

Control de gastos personales (100% local, sin internet)

Nuevo gasto

Concepto (ej. Comida, transporte...)

Comida

Monto (MXN)

Agregar gasto

Mis gastos

Total: \$338.50

Filtrar por categoría

Todas

Tacos con amigos

Comida • 2026-09-22

\$195.00

Editar

Eliminar

Camion a la universidad

Transporte • 2026-09-22

\$45.50

Editar

Eliminar

Fotocopias e impresiones

Escuela • 2026-09-22

\$60.00

Editar

Eliminar

Cafe con leche

Comida • 2026-09-22

\$38.00

Editar

Eliminar

Exportar a CSV

Figura 8. Gasto "Tacos con amigos" ya editado (su monto cambió de \$180 a \$195): el formulario vuelve a modo "Nuevo gasto" y el total general se recalcula a \$338.50.

ExpenseTrack Lite

Control de gastos personales (100% local, sin internet)

Nuevo gasto

Concepto (ej. Comida, transporte...)

Comida

▼

Monto (MXN)

Agregar gasto

Mis gastos

Total (Comida): \$233.00

Filtrar por categoría

Comida

▼

Tacos con amigos

Comida • 2026-09-22

\$195.00

Editar

Eliminar

Cafe con leche

Comida • 2026-09-22

\$38.00

Editar

Eliminar

Exportar a CSV

Plataforma: linux — Electron 31.7.7 — Chromium 126.0.6478.234

Figura 9. Función de filtro por categoría: al elegir "Comida" en el selector, la lista solo muestra los gastos de esa categoría y el total cambia a "Total (Comida): \$233.00".

Página 15 de 20

ExpenseTrack Lite

Control de gastos personales (100% local, sin internet)

Nuevo gasto

Concepto (ej. Comida, transporte...)

Comida

Monto (MXN)

Agregar gasto

Mis gastos

Total: \$338.50

Filtrar por categoría

Todas

Tacos con amigos

Comida • 2026-09-22

\$195.00

Editar

Eliminar

Camion a la universidad

Transporte • 2026-09-22

\$45.50

Editar

Eliminar

Fotocopias e impresiones

Escuela • 2026-09-22

\$60.00

Editar

Eliminar

Cafe con leche

Comida • 2026-09-22

\$38.00

Editar

Eliminar

Exportar a CSV

Exportado a: /root/expensetrack_gastos.csv

Figura 10. Función de exportar a CSV: al presionar el botón, la app genera el archivo expensetrack_gastos.csv en la carpeta de Documentos del equipo y muestra la ruta exacta donde quedó guardado.

Página 16 de 20

9. Tabla Comparativa: Requisitos de una Aplicación Multiplataforma vs. ExpenseTrack Lite

De la misma forma en que en los subproductos anteriores comparé mis proyectos contra lo que exige cada definición, aquí reviso, criterio por criterio, en qué va ExpenseTrack Lite respecto a lo que se espera de una aplicación multiplataforma real:

CRITERIO MULTIPLATAFORMA	QUÉ EXIGE LA DEFINICIÓN	¿CUMPLE?	EVIDENCIA EN EXPENSETRACK LITE
Un solo código fuente	Debe existir una única base de código compartida entre plataformas	Sí	Un solo proyecto Electron (main.js, index.html, renderer.js) sin ramas de código por sistema operativo
Corre en más de un sistema operativo	Debe poder ejecutarse en al menos dos sistemas operativos distintos	Sí	electron-builder está configurado para generar builds de Windows (nsis), macOS (dmg) y Linux (AppImage)
Interfaz consistente entre plataformas	La apariencia y el comportamiento deben ser equivalentes en cada plataforma	Sí	La interfaz HTML/CSS se renderiza igual con el motor Chromium embebido, sin importar el sistema operativo
Acceso a recursos del equipo	Debe poder interactuar con el sistema de archivos u otros recursos locales	Sí	Lee, escribe y edita gastos.json en la carpeta de datos del usuario, y exporta un .csv a la carpeta de Documentos, todo con el módulo fs de Node.js
Funciona sin conexión	Debe poder operar sin depender de internet	Sí	Toda la app corre y guarda datos localmente; no hace ninguna petición de red
Comunicación segura interfaz-sistema	Debe aislar el acceso a APIs del sistema para no exponer riesgos de seguridad	Sí	contextIsolation activado y contextBridge en preload.js; el renderer no tiene acceso directo a Node.js
Empaquetado distribuible	Debe poder generar un instalador o paquete final para el usuario	Parcial	Configuración de electron-builder lista; la generación de los tres instaladores finales queda como siguiente paso

De los siete criterios revisados, ExpenseTrack Lite cumple con seis de forma completa y uno está parcial: la configuración para generar los tres instaladores finales (Windows, macOS y Linux) ya existe en el proyecto, pero aún no generé y probé los tres paquetes finales en cada sistema operativo real, algo que dejo como siguiente paso fuera del alcance de este reporte.

10. Lista de Cotejo

Reviso, a manera de autoevaluación, que el documento cumpla con todos los puntos solicitados para esta actividad:

ELEMENTO SOLICITADO	CUMPLE	OBS.
Portada con formato oficial UTP	Sí	
Índice / tabla de contenido	Sí	
Introducción	Sí	
Definición de qué es una aplicación multiplataforma	Sí	
Ilustración del funcionamiento (diagrama)	Sí	
Características principales	Sí	
Ventajas y desventajas	Sí	
Comparativa (multiplataforma vs. móvil vs. web vs. escritorio)	Sí	
Tecnología utilizada	Sí	
Ejemplo de mi propia aplicación multiplataforma (ExpenseTrack Lite)	Sí	
Imágenes de la aplicación (evidencia propia)	Sí	Capturas reales de la app corriendo
Tabla comparativa: requisitos vs. mi aplicación	Sí	
URL de la aplicación (repositorio)	Sí	
Conclusión	Sí	
Referencias	Sí	

11. Conclusión

Las aplicaciones multiplataforma resuelven un problema muy concreto que fui identificando a lo largo de los cinco subproductos: escribir y mantener un código distinto por cada sistema operativo es costoso, y no siempre se justifica frente al beneficio de compartir una sola base de código. A cambio, como documenté en la comparativa, se paga un costo en rendimiento y en consumo de recursos, al depender de un runtime embebido en vez de compilar directamente a código nativo de cada plataforma.

Construir ExpenseTrack Lite con Electron me permitió aplicar directamente lo que aprendí en el Subproducto No. 2 (desarrollo web) a un contexto distinto: una aplicación de escritorio real, instalable, con acceso al sistema de archivos del equipo, en vez de depender de un navegador y una conexión a internet. Documentar sus capturas reales, corriendo como ventana de escritorio, hizo tangible la diferencia entre una página web y una aplicación multiplataforma empaquetada.

Con este quinto subproducto cierro un panorama completo de cinco enfoques de desarrollo de software —nativo (Wear OS), web (MediCore), escritorio (TaskDesk), móvil (PediatríaTrack Global) y multiplataforma (ExpenseTrack Lite)— documentados con el mismo tipo de análisis: definición, características, ventajas y desventajas, comparativa y tabla de cumplimiento. Esto me deja claro que no existe un enfoque superior en abstracto: la elección correcta depende de qué tanto pesa, para cada proyecto, el rendimiento, el acceso al hardware, el costo de desarrollo, el mantenimiento y la dependencia de internet.

12. URL de la Aplicación

Como evidencia del ejemplo presentado (ExpenseTrack Lite), se listan sus datos de acceso:

- **Repositorio de código fuente:** <https://github.com/AlexAP1115web/ExpenseTrackLite.git>

La aplicación aún no está publicada como instalador descargable: se ejecuta y se demuestra corriéndola directamente desde el código fuente. PASOS: al clonar el repositorio, dentro de la carpeta del proyecto ejecutar en una terminal con Node.js instalado:

```
npm install
```

```
npm start
```

Esto abre ExpenseTrack Lite como una ventana de escritorio nativa. Para generar los instaladores finales de cada sistema operativo (Windows, macOS y Linux) a partir del mismo código, el proyecto ya incluye la configuración de electron-builder, que se ejecuta con:

```
npm run electron-builder
```

Referencias

OpenJS Foundation. (2026). *Electron documentation*. <https://www.electronjs.org/docs/latest/>

Electron Userland. (2026). *electron-builder documentation*. <https://www.electron.build>

OpenJS Foundation. (2026). *Node.js documentation*. <https://nodejs.org/en/docs>

Google. (2026). *Chromium projects*. <https://www.chromium.org/chromium-projects/>

Google. (2026). *Flutter documentation*. <https://docs.flutter.dev>

Meta. (2026). *React Native documentation*. <https://reactnative.dev/docs>

Microsoft. (2026). *.NET MAUI documentation*. <https://learn.microsoft.com/dotnet/maui/>

Git. (2026). *Git documentation*. <https://git-scm.com/doc>